FAKTION
ARTIFICIAL INTELLIGENCE.
APPLIED.

SYMETA HYBRID · TECHNICAL TRAINING

Claude & Claude Code in your coding workflow

Full-day workshop · 16 July & 27 August 2026 · Thomas Claessens, Faktion

Increase performance · Spend fewer tokens · Keep improving on your own

AGENDA

One day, seven blocks, five labs

09:00	1	From chatting to delegating
09:45	2	An AI-ready codebase
11:00		*Break*
11:15	3	Context, scope & planning
12:15		*Lunch*
13:15	4	Skills — automate yourself
14:45		*Break*
15:00	5	From writer to reviewer
15:45	6	Power tools
16:30	7	Keep getting better
17:15		*Wrap-up*

BLOCK 01 · 09:00

From chatting to delegating

What changes when the model has hands — and why copy-paste is the most expensive way to use an LLM.

It only ever predicts the next token

1

One token at a time

Generates text one token — roughly ¾ of a word — at a time, each predicted from all the text before it. That is all it ever does, just very fast.

2

Stateless by design

Nothing persists between calls. Every turn re-sends the entire conversation — "memory" is just replaying the transcript back to the model.

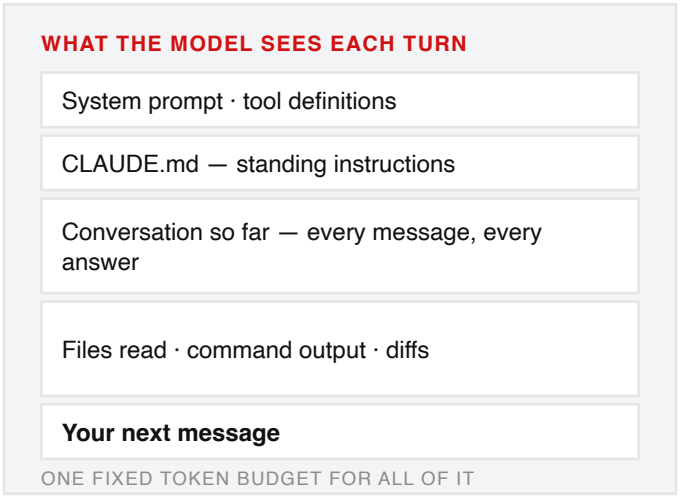
3

The system prompt

Standing instructions prepended before anything you type: role, rules, tool definitions, CLAUDE.md. Invisible in the UI, but it shapes every answer.

Text in, text out — nothing else. Everything the model knows about your task is text that you — or your tooling — put in front of it.

The context window: all it knows, every turn



You pay for the whole window, every turn

Each reply re-processes everything in the window. Cost and latency scale with its size — not with the size of your question.

Relevance beats volume

The model attends to everything it sees. Irrelevant files, stale chat and pasted dumps dilute the signal — answers drift, quality drops.

It runs out

Long sessions fill the window, and quality degrades before the hard limit. Block 3 covers `/clear` and `/compact` to manage it.

Small, relevant context = better, faster, cheaper. Every technique you learn today is, underneath, a way of managing this window.

The agent doesn't just answer — it acts

1

Reads & explores

Navigates your repo itself: files, structure, git history. No more pasting snippets — it fetches its own context, exactly what it needs.

2

Edits & creates

Changes files in place, across the codebase. You review diffs instead of copy-pasting blocks and hand-stitching them together.

3

Runs & verifies

Executes dotnet build, dotnet test, scripts. It sees its own mistakes and fixes them before you ever look.

Context window = its working memory. Tokens = what you pay, in money and speed. Everything today follows from these two facts.

Two interfaces, one engine

Terminal (CLI)

- Lives where your code lives — fastest feedback loop
- Slash commands, scripting, hooks, CI integration
- The power-user surface: everything works here

Desktop app (GUI)

- Same agent, friendlier surface — no terminal required
- Great on Windows: point it at a folder and go
- Zero-friction start if the CLI feels foreign today

Pick either — the habits transfer 1:1. Heads-up: some commands in this deck are CLI-native (`/cost`, `/usage`); the desktop app surfaces the same info in its UI instead.

Same bug, two ways

Chat + copy-paste

- You hunt for the right snippets to paste
- Model guesses at everything it can't see
- Every round-trip re-sends the whole story
- You hand-merge the answer back in — and hope

Claude Code in the repo

- Agent finds the failing code itself
- Reads exactly the files it needs — no more
- Runs the tests, sees them pass
- You review one diff

> Watch the `/cost` counter during the demo — better result, fewer tokens. This is the thesis of the day, shown live.

Prompt → Plan → Act → Verify

Prompt 01

Say what you want and why. Point at files, tickets, errors — context beats prose.

Plan 02

For anything non-trivial: plan mode first. Review the approach before any file changes.

Act 03

The agent edits and runs commands. The permission model keeps you in control of every write.

Verify 04

Build + tests run automatically. You review the diff — never blind-trust, never hand-merge.

Trust grows with the loop: start reviewing everything, loosen permissions as the repo setup (block 2) earns it.

Lab 0 — First contact

15 MIN · SOLO

1. Open the sample repo in Claude Code (CLI or desktop — your choice)
2. Ask it to explain the codebase: architecture, main flows, where the tests live
3. One test is failing. Ask the agent to find it, explain the cause, and fix it
4. Run `/cost` and note the number — we'll beat it after block 2

DONE WHEN

- Failing test is green
- You can explain the fix
- You wrote down your `/cost`

00

BLOCK 02 · 09:45

An AI-ready codebase

Output quality is mostly decided before you type the prompt. A prepared repo makes every future session better and cheaper.

CLAUDE.md — say it once, never again

WHAT BELONGS

```
# Payments API — CLAUDE.md

## Build & test
dotnet build · test · format
integration tests need the Docker DB

## Conventions
constructor DI, one handler per file

## Hard rules
never edit /Generated
no EF migrations without asking
```

What doesn't

- Anything the agent can read from the code itself
- Long prose — it's loaded every session, so every word pays rent
- Stale rules nobody enforces (they poison trust)
- One-off task instructions — those belong in the prompt

***`/init`** generates a starting point. It is never the end point — refining it is a habit, not a task.*

Give the agent eyes: build, test, analyze

1

dotnet build

Static typing is your superpower. Compile errors give the agent instant, precise, free feedback — every wrong guess dies in seconds, not in review.

2

dotnet test

Fast tests = the agent verifies its own work. A repo where the suite takes 20 minutes is a repo where the agent flies blind — invest in a fast core suite.

3

Analyzers & linters

Roslyn analyzers, dotnet format, nullable reference types. Every automated check is a reviewer the agent consults for free, before you have to.

Rule of thumb: anything a senior dev would check before approving a PR should be a command the agent can run.

What helps humans navigate helps agents more

- **Predictable layout.** Consistent project structure means the agent finds things instead of searching for them — search costs tokens.
- **Deep modules, small files.** A 2,000-line god-class must be read whole; ten focused files are read selectively.
- **Names from the shared language.** `CONTEXT.md` terms become class, method and test names — navigation gets cheaper for agent and human alike.
- **Permissions per repo.** Tight on the legacy monolith, looser on the greenfield service.

TEAM INFRASTRUCTURE

Check **.claude/ into git.**
CLAUDE.md, skills, hooks and settings are code — version them, review them, share them. One dev's improvement becomes everyone's.

Hooks — rules that enforce themselves

Shell commands that fire on agent lifecycle events — before a tool runs, after an edit, on finish. Deterministic, not optional.

1

Block

Dangerous commands refused before they run: git push --force, reset --hard, clean. The agent physically can't — no vigilance required.

2

Enforce

Auto-run dotnet format after every edit. Require green tests before any commit. Quality gates the agent passes through, every time.

3

Notify

Ping you when the agent finishes, gets stuck, or needs a decision. Kick off long tasks and actually walk away.

CLAUDE.md asks. Hooks enforce. Move anything safety-critical from prose into a hook — and version hooks in git like everything else in .claude/.

When hooks fire — the session lifecycle

They fire at fixed points in the agent's loop; a matcher narrows each to the tools or triggers you care about.

1

PreToolUse

Fires before a tool runs — and can block it. Matcher = tool name (Bash, Edit|Write). Exit 2 refuses the call.

2

PostToolUse

Fires after a tool succeeds. Same tool-name matcher. Auto-format, run tests, or verify what just changed.

3

SessionEnd

Fires when the session ends. No blocking — fire-and-forget: clean up scratch files, log the run, archive it.

Match the event to the moment. SessionStart seeds context at launch; PreToolUse guards, PostToolUse verifies, SessionEnd cleans up.

Three hooks you'll actually write

.claude/settings.json

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash", "hooks": [{ "type": "command",
          "if": "Bash(git push --force*)",
          "command": "echo blocked >&2; exit 2" }] }],
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [{ "type": "command",
          "command": "dotnet format" }] }],
    "SessionEnd": [
      { "hooks": [{ "type": "command",
          "command": "./.claude/hooks/wrap-up.sh" }] }]
  }
}
```

- **PreToolUse** runs before the call and can veto it — a force-push never reaches git. Your safety rail.
- **PostToolUse** runs after an edit lands. dotnet format here; swap in dotnet test to gate commits.
- **SessionEnd** has no matcher and can't block — ideal for cleanup and logging as you exit.

One settings.json in .claude/, versioned in git — the whole team inherits the same guardrails.

Lab 1 — Make the repo AI-ready

40 MIN · PAIRS

1. Run `/init` on the sample repo, then cut the generated `CLAUDE.md` down to what earns its place
2. Add a few hooks, perhaps you can think of another one to implement? Review the documentation and come up with an idea!
3. Start a `CONTEXT.md`
4. Ask the agent to implement a new feature and ask it to update the `CONTEXT.md`

DONE WHEN

- `CLAUDE.md` < 60 lines, all of it true
- `Hooks` have been added and are working
- New feature is implemented, Context.MD is updated

01

BLOCK 03 · 11:15

Context, scope & planning

Working effectively inside a session: keep context clean, cut work small, and align before any code gets written.

A conversation is a consumable

1

One task, one session

Context accumulated for task A is noise (and cost) for task B. Finish, `/clear`, start clean. Long meandering sessions degrade quality and burn tokens.

2

/clear vs /compact

`/clear` wipes the slate — default between tasks. `/compact` summarizes when you must continue past a long history. Know which you need before the window fills.

3

Don't hoard context

Point at files instead of pasting them. Reference tickets by ID. Let the agent pull what it needs — pulling the right thing beats pushing everything.

Symptom check: if you're scrolling to remember what the session was about, the agent is struggling too.

Handoffs — context as a file, not a scrollback

Have the agent distill the session into a document: goal, findings, decisions, next steps. Then anything can pick it up.

1

Continue fresh

Session getting heavy? Handoff → `/clear` → continue from the doc. One distilled page beats 50k tokens of history — same task, fraction of the cost.

2

Cross-model relay

Let the expensive model think, the cheap model type: plan or review on the top model, hand the doc to a cheap model to execute.

3

Cross-agent pipelines

Review a PR, write the findings into a handoff doc — a second agent implements the changes. The doc is the interface between agents.

Bonus: a handoff doc is reviewable and versionable. You can read, correct and commit what gets carried forward — you can't do that with a context window.

Get grilled before a line is written

- **The #1 failure mode is misalignment,** not capability. You think it understood; it built something else. Same as with a human dev — nobody knows exactly what they want.
- **Flip the interview.** Ask the agent to grill you: relentless questions until every branch of the decision tree is resolved (Pocock's `/grill-me` pattern).
- **Plan mode for anything > 1 file.** Review the plan, correct it cheaply in prose — not expensively in code review.

THE MATH

10 minutes of questions

costs less than

one wrong implementation

— in tokens, in time, and in your patience.

Plan expensive, implement cheap

Top model → the plan

- Grilling session + plan mode on the strongest model
- This is where judgment lives: trade-offs, edge cases, design
- Output: a tight, unambiguous plan

Cheap model → the code

- The relay: handoff → `/clear` → cheap model implements from the doc
- Don't just `/model` mid-session: the new model drags the full transcript along — more tokens, not fewer
- A precise plan makes a cheap model perform like an expensive one

This is the single biggest cost lever you'll learn today. Judgment is scarce and expensive; execution against a good plan is not.

Git worktrees — parallel universes, one repo

`claude --worktree feature-b` — Claude creates a working folder on its own branch and drops you in it. No manual git worktree add, no stashing, no juggling.

1

Parallel agents

Run two agent sessions at once without them stomping each other's files — one fixes a bug while another builds a feature.

2

Compare approaches

Same problem, two worktrees, two strategies. Build both, keep the winner, delete the loser. Today's Lab 2 runs exactly like this.

3

Clean review space

The agent churns in its worktree; your own checkout stays untouched for review, hotfixes, and honest diffs.

Agents make parallel work cheap — worktrees make it safe. They're the natural pairing.

Lab 2 — One-shot vs. aligned

60 MIN · SOLO · BOTH SCENARIOS

1. Write BOTH prompts up front: one-shot + grilling kickoff. No edits after. Target: both backlog features — edit a todo, and filter the list
2. Let Claude own the worktrees — start each round with `claude --worktree lab2-oneshot` and `claude --worktree lab2-plan`. It creates and enters each one. No manual git
3. Round 1 in lab2-oneshot: fire the one-shot prompt on the default model. Take what you get
4. Round 2 in lab2-plan: grill on the top model → handoff → `/clear` → implement on a cheap model
5. Compare your own two results: correctness, code quality, total `/cost`. Numbers on the board

DONE WHEN

- Both versions build and run
- You compared your own pair honestly
- Costs for both rounds on the board

02

BLOCK 04 · 13:15

Skills — automate yourself

The heart of the day: noticing repetition, and turning it into tooling. Not a framework to adopt — a reflex to build.

What a skill actually looks like

.claude/skills/deslop/SKILL.md

```
---
name: deslop
description: Clean AI-slop from code and prose.
  Use when the user asks to "deslop", or after
  generating a large amount of code — strip filler
  comments, over-abstraction, hedging prose.
---

1. Remove comments that restate the code.
2. Inline any abstraction with a single caller.
3. Delete hedging language from docs and messages.
4. Report what you removed, in one line.
```

- **name** becomes the slash command: **`/deslop`**.
- **description** is the only part that's always in context — it's the trigger surface. The "Use when…" phrases (gold) are what the model matches against your request.
- **The body** loads only when triggered — progressive disclosure. Opinions and steps live here, token-free until needed.

It graduates to a skill when you've pasted the same instructions twice.

Two ways in — and how to control them

User-invoked

You type `/deslop`. Deliberate, predictable — always available for any skill.

Model-invoked

You say “clean this up” — the agent matches the description and loads the skill itself. No command typed.

```
disable-model-invocation: true
```

Slash-command only — the model can never trigger it on its own. Use for anything with side effects: deploy, commit, send a message. You control the timing.

```
allowed-tools: Bash(git *), Read
```

Pre-approves the listed tools while the skill runs — no permission prompts mid-flow. Everything else still follows your normal permission rules.

*This is called **Frontmatter** - You can find the docs here! - There are more examples and they are specific to the harness aswel!*

Rules vs. commands vs. skills

ARTIFACT	WHAT IT ENCODES	WHEN LOADED	USE FOR
Rule (CLAUDE.md)	A fact or constraint — always true	Every session, every request	Conventions, hard limits, gotchas — keep it lean, every word pays rent
Command	A fixed, frequent action	When you type it	Deterministic shortcuts — no judgment involved
Skill	A process with steps, opinions & judgment	Only when relevant (progressive disclosure)	Repeatable workflows: reviews, debugging, ticket writing, releases

Rule of thumb: fact → CLAUDE.md · action → command · process → skill

Reading the masters: Pocock's library

PROCESS

/tdd

Encodes red-green-refactor and opinions on what makes a good test. The agent doesn't just write tests — it follows a discipline.

DISCIPLINE

/diagnosing-bugs

Reproduce → minimise → hypothesise → instrument → fix → regression-test. Debugging as a loop, not a vibe.

CONVENTION

/code-review

Uses concepts based on Matt Fowler's **Refactoring** book and puts it into a skill you can use on the fly.

Steal the patterns, not the files. Good skills are small, composable, opinionated — and encode conventions as much as technique.

Skills compose — small parts, real pipelines

Because skills are just instructions, a skill can invoke other skills. Example: a `/pr-review-handoff` skill —

01

Read the PR

Pull the diff, the description, the linked ticket. Understand what's actually being changed and why.

›

02

Grill you

The grilling pattern, reused: what's risky here, what's out of scope, which standards apply to this change?

›

03

Write the handoff

Findings and required changes distilled into a handoff document — the reusable interface between agents.

›

04

Implement fresh

A clean session (cheap model!) picks up the doc and makes the changes. Review judgment and typing decoupled.

Each part is a skill you already have. Composition is where a personal toolkit becomes a team workflow.

Did it twice? That's a skill.

Repetition is the trigger.

You pasted the same instructions again → **skill**

You explained the same process again → **skill**

You fixed the same kind of mistake again → **skill (or a rule)**

You ran the same 5-step dance again → **skill**

Lab 3 — Write your own skill

60 MIN · SOLO, THEN SHARE-OUT

1. Pick a recurring pain point from YOUR daily work — code review conventions, ticket writing, release notes, a Symeta-specific process
2. Use Pocock's `/write-a-skill` to build it: structure, progressive disclosure, a description written for the model
3. Stuck for ideas? Build a deslop skill: strip AI-slop from generated code — filler comments, over-abstraction, hedging prose
4. Test it on real material. Iterate once. Volunteers demo at the end

DONE WHEN

- Skill triggers correctly
- It encodes YOUR convention, not a generic one
- You'd actually use it next week

03

BLOCK 05 · 15:00

From writer to reviewer

It’s not ‘vibe’ coding if you utilize the harness with a plan in the front and a review in the back.

BLOCK 05 · THE ROLE SHIFT

You are now a reviewer

WHAT CHANGED

You read more than you write

The agent produces a day of diffs in minutes — reading code is now the job, writing it the exception. Your throughput is judgment speed, not typing speed.

WHAT DIDN'T

You own what you merge

"Claude wrote it" is not a code review — accountability doesn't delegate. If you can't explain the diff, you can't merge it.

THE TRAP

Plausible ≠ correct

AI code is optimized to look right — which defeats skimming. Human bugs cluster in hard places; AI bugs hide in confident, clean-looking code.

Review is now the bottleneck. The last skill of today: reviewing fast without reviewing carelessly.

The slop checklist — what AI code smells like

Vacuous tests

Green but asserts nothing: Assert.NotNull on a non-nullable, mocks verifying mocks, no failing case anywhere.

Speculative abstraction

An interface with one implementation, a factory for a constructor — built for a future nobody asked for.

Swallowed failures

`catch (Exception) { log }` — and execution just continues. Errors handled by hiding them.

Silent behavior change

A "refactor" that also changes rounding, casing or defaults. Diff the behavior, not the style.

Reinvented framework

Hand-rolled JSON parsing next to System.Text.Json; a bespoke retry loop instead of Polly.

Comment noise

`// increment the counter` — narrates what, not why. Delete on sight.

Encode this list as your team's `/review` skill. The agent runs it on its own output before you ever look.

Reviewing at generation speed

1

Agent pre-reviews

Your `/review` skill — slop checklist plus team conventions — runs on the diff before human eyes. A fresh session reviews better than the one that wrote it.

2

Tests first, run them

Tests are the diff's claim about itself — if they're wrong, stop reading. Run them; break the code once and watch a test fail.

3

Interrogate the diff

Ask what changed in behavior vs. style; make it defend edge cases. If a fix costs more than re-prompting: `/clear` and re-roll.

BLOCK 06 · 15:45

Power tools

The big guns you won't use every day. Know they exist, reach for them when the problem fits — they multiply capability and cost.

MCP servers — give the agent more tools

- **What it is:** an open protocol that plugs external systems into the agent as tools — Jira, Azure DevOps, Confluence, databases, browsers.
- **Why it beats copy-paste:** the agent queries live data itself, mid-task. 'Implement PROJ-142' — it reads the ticket, spec and comments. No ferrying.
- **Team angle:** the config lives in the repo — set up once, the whole team benefits.

```
.mcp.json — committed to the repo

{
  "mcpServers": {
    "issue-tracker": { "type": "http",
      "url": "https://<your-mcp-host>/mcp" }
  }
}
```

LIVE DEMO

Ticket → implementation

Agent reads the work item from your tracker, plans against the codebase, and links the PR back — without you pasting a word.

For when a normal session isn't enough

Subagents

Fan a large task across parallel agents, each with an isolated context. Codebase-wide audits, big refactors, mass migrations.

ultracode

Max reasoning effort + automatic multi-agent orchestration in one setting (`/effort ultracode`). For the gnarly problem that survived a normal session.

/loop

Run a prompt on an interval — an in-session cron. 'Every 5m: check the deploy, ping me if it goes red.'

Headless / CI

Claude Code inside pipelines: automated PR review, changelog generation, scheduled maintenance chores.

Cost caveat: these multiply token usage. The tiering principle applies — expensive tools for expensive problems, never as the default.

BLOCK 07 · 16:30

Keep getting better

Today ends; the improvement loop doesn't. Treat your AI workflow like a codebase — it needs maintenance and refactoring.

Every bad session points at a fixable artifact

01

Bad session

(or a surprisingly good one). Don't shrug and move on — that's data.

02

Ask why

Missing context? Repeated yourself? Misunderstood the domain? Wrong scope?

03

Fix the artifact

Missing context → `CLAUDE.md`. Repetition → skill. Domain confusion → `CONTEXT.md`. Too big → slice smaller.

04

It compounds

Every future session inherits the fix. Watch `/cost` trend down over weeks.

This loop is the actual deliverable of today. Everything else was an instance of it.

Individual reflexes → team capability

1

Share the tooling

Skills, `CLAUDE.md` and hooks live in git. One dev's improvement lands in everyone's next session. Review skill changes like code.

2

#claude-tips

A channel where 'look what worked' gets posted. Low ceremony, high compounding. Seed it yourself for two weeks.

3

Monthly workflow retro

30 minutes: each dev demos one improvement — a skill, a hook, a `/cost` win. The loop, made social.

Staying current: the Claude Code changelog, community skill libraries (Pocock's among them) — evaluate new features skeptically, adopt what survives the retro.

Sharing is Caring

1

Experiment

Saw something online? New model? New Skill (Framework)? Try it out!

Don’t base your conclusions on one experiment though!

2

Share

Talk to your colleagues, post your findings, share interesting articles and newsletters.

3

Implement

Have your AI Champion distribute your findings through your AI framework / skill library / plugin / ruleset….

A new model or workflow is **litterally** being released every other **week**. New isn’t always better, so be cautious.

BEFORE YOU LEAVE

Write down two commitments

1 The first repo you'll make AI-ready next week — CLAUDE.md, feedback loops, CONTEXT.md.

2 The first skill you'll write — the thing you explained twice this month.

Questions → Thomas Claessens · t.claessens@faktion.com

The reflex, not the library. Did it twice? That's a skill.